

C OPERATORS

The Logic and Math of C Programming



Comprehensive 14-Page Technical Guide

01. Concept of Operators

In C, operators are special symbols that instruct the compiler to perform specific mathematical or logical manipulations. They act upon variables and values, which are known as **Operands**.

Definition: An operator is a symbol that represents an action. For example, '+' is an operator that represents addition.

Operators are the building blocks of expressions and algorithms. Without them, variables would simply be static data storage with no way to interact.

02. Arithmetic Operators

Used for fundamental mathematical calculations. These are Binary Operators as they require two operands.

Symbol	Operation	Example (a=10, b=3)
+	Addition	$a + b = 13$
-	Subtraction	$a - b = 7$
*	Multiplication	$a * b = 30$
/	Division	$a / b = 3$ (Integer)
%	Modulus	$a \% b = 1$ (Remainder)

Note: Modulus (%) only works with integers!

03. Relational Operators

Used to verify the relationship between two operands. They return a boolean value: **1 (True)** or **0 (False)**.

- **==** : Equal to
- **!=** : Not equal to
- **>** : Greater than
- **<** : Less than
- **>=** : Greater than or equal to
- **<=** : Less than or equal to

Example: (5 == 3) results in 0.

04. Logical Operators

Used to combine multiple relational expressions or to negate a condition.

&& (Logical AND)

Returns True only if **both** operands are True.

|| (Logical OR)

Returns True if **at least one** operand is True.

! (Logical NOT)

Reverses the logical state (True becomes False).

05. Assignment Operators

Used to store values in variables. C provides "Compound Assignment" operators for shorthand notation.

Operator	Example	Equivalent to
=	a = 5	a = 5
+=	a += 2	a = a + 2
-=	a -= 2	a = a - 2
*=	a *= 2	a = a * 2
/=	a /= 2	a = a / 2

06. Unary Operators (++ , --)

These operators add or subtract exactly 1 from the variable's value.

Two Flavours:

Prefix (++a): Increases value first, then uses it in the expression.

Postfix (a++): Uses current value in expression first, then increases it.

```
int x=5; printf("%d", x++); prints 5, but x becomes 6.
```

07. The Ternary Operator

The only operator in C that takes three operands. It is a shorthand for `if-else`.

```
Syntax: (condition) ? value_if_true : value_if_false;
```

Example: `(age >= 18) ? "Adult" : "Minor";`

08. Bitwise Operators

Operate on the actual bits (0s and 1s) of a number. Crucial for hardware programming.

- **&** : Bitwise AND
- **|** : Bitwise OR
- **^** : Bitwise XOR
- **~** : One's Complement

09. Bitwise Shifts

Moving bits to the left or right can effectively multiply or divide by 2.

Left Shift (<<): $5 \ll 1$ (binary 101 becomes 1010, which is 10).

Right Shift (>>): $10 \gg 1$ (binary 1010 becomes 101, which is 5).

10. Special Operators

- **sizeof:** Returns the size of a data type in bytes.
- **& (Address-of):** Returns the memory address of a variable.
- *** (Pointer):** Used to access value at a specific address (Dereference).
- **, (Comma):** Used to link multiple expressions together.

11. Precedence & Associativity

When multiple operators appear in one expression, C follows a strict order of operations.

1. Parentheses ()
2. Unary ++, --, !
3. Multiplication / Division *, /, %
4. Addition / Subtraction +, -

If operators have the same level, C uses **Associativity** (usually Left to Right).

12. Pitfalls to Avoid

- **= vs ==**: `if(x = 5)` will always be true (assignment), use `==` for comparison.
- **Integer Division**: `5/2` is 2. Use floats if you need 2.5.
- **Modulus with floats**: Using `%` on a float results in a compile error.

13. Operator Review

- **Arithmetic:** +, -, *, /, %
- **Relational:** ==, !=, >, <, >=, <=
- **Logical:** &&, ||, !
- **Assignment:** =, +=, -=, *=, /=
- **Unary:** ++, --, sizeof

Mastering these symbols is the key to mathematical logic in C.



END OF OPERATORS GUIDE